



An interpretable DeePMD-kit performance model for emerging supercomputers

Xiangyu Meng¹ · Xun Wang^{1,2} · Mingzhen Li^{2,3} · Guangming Tan^{2,3} · Weile Jia^{2,3}

Received: 22 July 2024 / Accepted: 10 November 2024 / Published online: 17 February 2025
© The Author(s) 2025

Abstract

Deep potential (DP) scheme has increased the simulation temporal and spatial scales while maintaining the ab initio accuracy of the molecular dynamics. DeePMD-kit is an outstanding application that implements DP scheme efficiently. However, current performance model cannot accurately measure the resource utilization of DeePMD-kit operators and predict the execution time. We introduce DP-perf, an interpretable performance model for DeePMD-kit. DP-perf can accurately measure the resource utilization of the individual DeePMD-kit operators, communication pattern, and the overall application by exploiting physical system properties and machine configurations. It can be easily applied to mainstream supercomputers including Tianhe-3F, the new Sunway, Fugaku, and Summit. With DP-perf, users can select the optimal machine and decide the corresponding configuration for various purposes (e.g., lower cost, less time) without real runs. Evaluation of four top supercomputers shows that DP-perf can fit overall execution time with a low mean absolute percentage error of 5.7%/8.1%/14.3%/13.1% on Tianhe-3F/new Sunway/Fugaku/Summit. On the prediction scenario, DP-perf can predict the total execution time with a mean absolute percentage error of less than 20%.

Keywords Performance model · Molecular dynamics · Deep potential · Performance optimization

1 Introduction

In recent years, molecular dynamics with ab initio accuracy have provided tools for the microscopic understanding of various phenomena in many different disciplines, including but not limited to chemistry, biology, and materials science.

However, the spatial and temporal scales are limited (can only deal with hundreds of atoms with picoseconds) due to the complexity of the ab initio molecular dynamics (AIMD) method. Although some linear methods have achieved millions of atoms with less than hundreds of femtoseconds per day on top supercomputers, they are limited to insulating systems with large band gaps (Lin et al. 2009; Hu et al. 2022; Schade et al. 2022; Yan et al. 2023). Meanwhile, empirical force fields (EFFs) methods (Richards et al. 2009; Salomon-Ferrer et al. 2013; Van Duin et al. 2001) have enabled many practical interests including heterogeneous catalysis, electrochemical cells, and biomedical reactions requiring trillion atoms or even larger simulations. However, EFFs cannot accurately describe quantum-mechanical phenomena such as bond breaking, complex chemical processes, quantum dots, etc.

Neural network molecular dynamics (NNMD) has received significant research interest due to its ability to balance ab initio accuracy with computational efficiency (Friederich et al. 2021; Unke et al. 2021). High-dimensional nonlinear potential energy surfaces (PES) can be accurately fitted by leveraging neural networks and extensive training data. It has been developed by the

✉ Xun Wang
wangsyun@upc.edu.cn

✉ Mingzhen Li
limingzhen@ict.ac.cn

✉ Weile Jia
jiaweile@ict.ac.cn

Xiangyu Meng
x.meng@s.upc.edu.cn

Guangming Tan
tgm@ict.ac.cn

¹ Department of Computer Science and Technology, China University of Petroleum, Qingdao, China

² State Key Lab of Processors, Institute of Computing Technology, Beijing, China

³ University of Chinese Academy of Sciences, Beijing, China

Behler–Parrinello scheme in 2007 (Behler and Parrinello 2007), which brings the foundation for the development of various NNMD packages [e.g., NEquIP (Batzner et al. 2022), PaiNN (Schütt et al. 2021), and the Deep Potential (DP) scheme (Zhang et al. 2018; Wang et al. 2018; Zhang et al. 2018, 2020, 2022)]. DeePMD-kit is an exemplary implementation of DP scheme, achieving unprecedented simulations of up to 10 billion atoms with nanoseconds per day performance on top supercomputers (Guo et al. 2022; Jia et al. 2020), which makes it the most popular package in the NNMD field for large-scale molecular dynamics simulations.

Current supercomputers are typically equipped with many-core chips and designed with innovative computing elements to improve the computational potential. These designs offer new opportunities to enhance the temporal and spatial scales of the DeePMD-kit. However, the performance gap between DeePMD-kit and the emerging supercomputers still exists. A performance model for DeePMD-kit is required to accurately identify the performance bottleneck and guide the researchers for better software–hardware co-design. Currently, numerous performance models have been designed and employed to analyze the performance of various applications or algorithms. Typical works include performance models to guide automatic parameter tuning (Tan et al. 2008; Xie et al. 2019), application-level performance models used to analyze computation time and machine computing capabilities (Jia et al. 2013; Williams 2009; Zhang et al. 2017), and communication-level performance models used to analyze communication bandwidth and network latency of supercomputers (Culler et al. 1993; Alexandrov et al. 1995). However, DeePMD-kit is a complicated application that contains many irregular customized operators and complex MPI communications. Current performance models cannot accurately characterize the performance utilization of these special operators and communication patterns in the DeePMD-kit. Moreover, opportunities to access the entire supercomputer are quite limited, and there is no effective approach to accurately estimate the simulation latency and the achieved peak performance of DeePMD-kit without executing real MD simulations on the whole machine. Besides, the current performance models have poor transferability and ignore the influence of simulated physical systems and computing platforms on DeePMD-kit performance.

In this paper, we introduce DP-perf, an interpretable DeePMD-kit performance model to analyze the performance

gap between MD simulation and emerging supercomputers in detail. Specifically, the main contributions are as follows:

- We leverage the characteristics of molecular systems and the DeePMD-kit algorithm to characterize the arithmetic workloads and memory access for each DeePMD-kit operator and the communication pattern.
So that the characterized workloads can be quickly and accurately calculated in response to changes of the simulated molecular system.
- We leverage the machine configurations (i.e. peak performance, memory bandwidth, and network bandwidth) to construct DP-perf performance model, which provides stable transferability across different supercomputers.
- We evaluate DP-perf on physical MD systems across four top supercomputers. Experimental results show that DP-perf can provide accurate performance prediction on both individual operators and the holistic simulation, which helps in analyzing the simulation efficiency and the simulation cost.

The rest of this paper is organized as follows. Section 2 describes the background and motivations. Section 3 introduces the DP-perf performance model. Section 5 presents the related works. Section 4 evaluates the accuracy of DP-perf. Section 6 concludes our work.

2 Background

2.1 DeePMD-kit framework

DeePMD-kit is an open-source NNMD package based on the DP scheme (Thompson et al. 2022). The overview of the computation procedure is illustrated in Fig. 1a, which includes two components: a descriptor $\mathcal{D}$ and a fitting net $\mathcal{N}$. $\mathcal{D}$ is used to calculate symmetry-preserving features from the input environment matrix $\tilde{R}_i$. $\mathcal{N}$ learns the relationship between the local environment features obtained from $\mathcal{D}$ and the atomic energy E_i . As a result, the PES of the whole system is expressed as the sum of the atomic energy contribution: $E = \sum_i E_i$

Figure 1b shows the computation of $\mathcal{D}$, which is denoted as

$$\mathcal{D}(\tilde{R}_i) = (\mathcal{G}_i^<)^\top \tilde{R}_i (\tilde{R}_i)^\top \mathcal{G}_i, \quad (1)$$

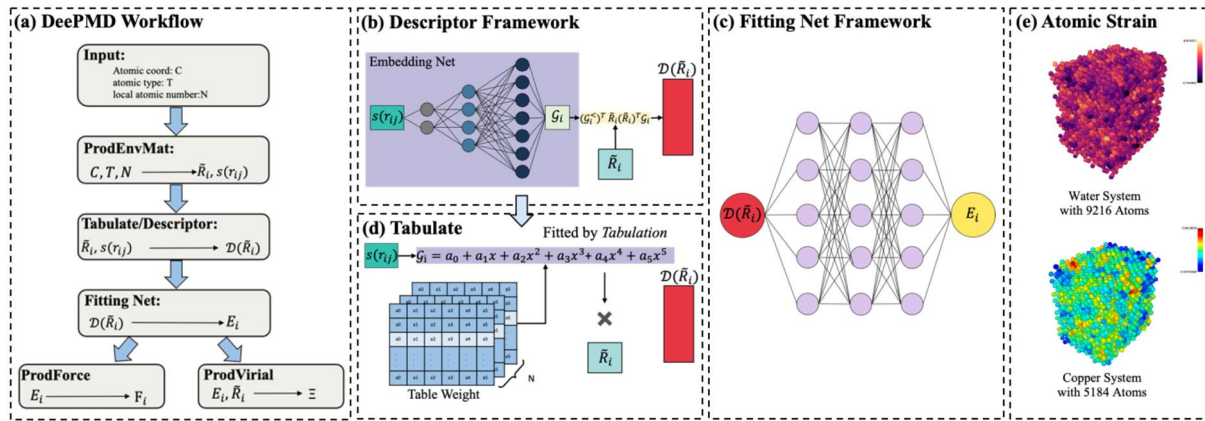


Fig. 1 Overview of DeePMD. **a** The computation workflow of DeePMD. **b** The descriptor network maps $s(r_{ij})$ and $\tilde{R}_i$ to $\mathcal{D}(\tilde{R}_i)$. It learns the physical symmetry features during the training stage. **c** The fitting net framework. It predicts atomic energy E_i from $\mathcal{D}(\tilde{R}_i)$.

d The framework of Tabulate operator. This operator compresses the descriptor network using Weierstrass approximation. **e** Perform ab initio molecular dynamics and analyze the physical or chemical properties of simulation regions

where $\mathcal{G}_i \in \mathbb{R}^{N_m \times M}$ is the embedding matrix, $\mathcal{G}_i^< \in \mathbb{R}^{N_m \times M^<}$ is a sub-matrix of $\mathcal{G}_i$ constructed by taking the first $M^<$ columns of $\mathcal{G}_i$, and the environment matrix $\tilde{R}_i$ is denoted as

$$\tilde{R}_i = s(r_{ij}) \times \left(1, \frac{x_{ij}}{|r_{ij}|}, \frac{y_{ij}}{|r_{ij}|}, \frac{z_{ij}}{|r_{ij}|} \right), \quad (2)$$

where $r_{ij} = r_j - r_i$ is the relative positions of each neighbor atom, and $s(r_{ij}) = \omega(|r_{ij}|)/|r_{ij}|$, which is a gating function that decays smoothly from 1 to 0 at $|r_{ij}| < r_c$. Matrix $\mathcal{G}_i$ is derived by an embedding network with the input of $s(r_{ij})$. The embedding net is a multi-layer residual fully connected model. In each layer, the output size is doubled and the output size of the model is M .

As shown in Fig. 1c, the fitting network $\mathcal{N}$ is a fully connected network that predicts E_i . Each layer is a fully connected layer with Tanh activation. Combining $\mathcal{D}$ and $\mathcal{N}$, DeePMD-kit can realize to learn representation from the environment matrix to the system PES. The force of atom i is calculated by the gradient of the back-propagation of the DP model, which is expressed as $F_i = -\nabla_{r_i} E = -\sum_j \nabla_{r_i} E_j$, where $\nabla_{r_i} E_j$ is calculated by the backpropagation. The virial of the system is denoted as $\Xi = \sum_i R_i F_i$, which is calculated through ProdVirial operator.

The calculations of the embedding network within the descriptor consume more than 90% of the total computation time. To reduce the computational burden, the *tabulation* method (Fig. 1d) (Guo et al. 2022) is proposed to approximate the embedding network, which is denoted as

$$\mathcal{G}_i = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5, \quad (3)$$

where $x = s(r_{ij})$. It is constructed by Tabulate and TabulateGrad operators to fit the weight of the embedding net after training using fifth-order polynomials through Weierstrass approximation.

2.2 Emerging supercomputers

We select four top supercomputers (i.e. Tianhe-3F, new Sunway, Fugaku, and Summit) to evaluate our DP-perf, as we can access the already published performance results or the open-source DeePMD-kit codes of these supercomputers. These platforms include homogeneous ARMv8 architecture, MPE-CPE heterogeneous architecture, and CPU-GPU heterogeneous architecture, which can represent the major supercomputing systems in the world. Table 1 shows the system overview of four supercomputers.

Table 1 System overview of four supercomputers

Supercomputer	Arch	Peak performance	Bandwidth	Total memory	Number of nodes
Tianhe-3F	ARMv8	2.9 PFLOPS	204.8 GB/s	625 PB	5000
New Sunway	—	1.5 EFLOPS	307 GB/s	10,080 PB	107,520
Fugaku	ARMv8	537 PFLOPS	1024 GB/s	4968 PB	158,976
Summit	X86	200 PFLOPS	900 GB/s	250 PB	9216

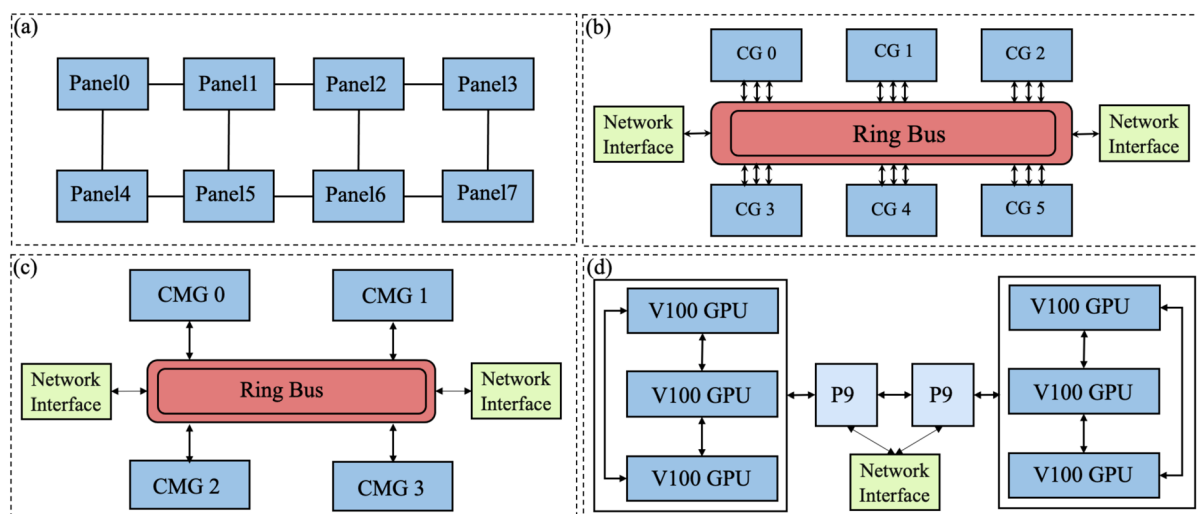


Fig. 2 Processors of the four top supercomputers. **a** Each Tianhe-3F node is equipped with an FT-2000+ processor containing eight NUMA domains called panels. Each panel has eight ARMv8 cores with 128-bit SIMD support. **b** Each new Sunway node is equipped with an SW26010-pro processor containing six NUMA domains called core groups (CGs). Each CG contains one management process element (MPE) used for management and IO and 64

computing process elements (CPEs) used for computation. **c** Each Fugaku node is equipped with an A64FX processor containing four NUMA domains called core memory groups (CMG). Each CMG contains 13 cores, one for operating system and IO, others for computing. **d** Each Summit AC922 node contains two IBM POWER9 processors and six NVIDIA Tesla V100 accelerators connected via NVLINK

The Tianhe-3 prototype is composed of FT-2000+ processors and MT-2000+ processors (You et al. 2019; Chen et al. 2021; Wang et al. 2020; Gan et al. 2021; Wei et al. 2021). The FT-2000+ queue of the Tianhe-3 prototype is called the Tianhe-3F supercomputer, which consists of 5000 computing nodes with 2.9 PFLOPS theoretical performance in double-precision. Each node in Tianhe-3F is equipped with an FT-2000+ processor shown in Fig. 2a, which adopts ARMv8 architecture and has 128-bit SIMD units offering a theoretical double-precision peak performance of 588.8 GFLOPS. Each FT-2000+ consists of eight NUMA structures called panels sharing 128 GB memory with 204.8 GB/s memory bandwidth. Each panel has an independent memory control unit (MCU) and eight cores. In each panel, each core has a 32KB L1 cache and 4 cores share an independent 2MB L2 cache. Meanwhile, each computing node interconnects with a bidirectional network with a bandwidth of 200 Gbps.

The new Sunway supercomputer contains 107,520 computing nodes and has a theoretical double-precision peak performance of 1.5 EFLOPS. The computing nodes are interconnected through a fat tree network topology, and every 256 nodes in a rack form a supernode (Gao et al. 2021; Zhu et al. 2021). Each node is equipped with an SW26010-pro processor shown in Fig. 2b, which contains 6 core groups (CGs) offering a theoretical double-precision peak performance of 14 TFLOPS. Each CG contains one management

pressing element (MPE) and an 8×8 computing processing element (CPE) grid connected to 16 GB DDR4 main memory with 51.2 GB/s bandwidth. Each CPE has a 256 KB manually controlled scratchpad memory called LDM. MPE and CPE have 256-bit and 512-bit SIMD units, respectively. The SW26010-pro processor supports direct memory access (DMA) to transfer contiguous data between LDM and main memory with a theoretical bandwidth of 307 GB/s. In addition, the data transfer among CPEs within the same CG has around 400 GB/s bandwidth through remote memory access (RMA). The programming model of the new Sunway is Sunway Accelerate Computing Architecture (SACA), which provides the basic compilation environment, the basic libraries, and the runtime.

The Fugaku supercomputer has a theoretical peak performance of 537 PFLOPS in double precision. It consists of 158,976 nodes and each node is equipped with an A64FX processor shown in Fig. 2c, which consists of 52 cores with 4 dedicated for operating system and IO as well as 48 for computation. The chip is organized with 4 NUMA domains called core memory groups (CMG), with each CMG connected to its 8 GB local HBM2 memory with 256 GB/s bandwidth. The A64FX processor supports 512-bit SVE operations coupled with two SIMD pipelines, enabling each core to execute 32 double-precision floating-point calculations per cycle. Meanwhile, Fugaku is equipped with a 6D

torus/mesh interconnect called a Tofu-D network with a theoretical bandwidth of 400 Gbps.

The Summit supercomputer can achieve around 200 PFLOPS in double precision. Meanwhile, its mixed precision capability can achieve 3 EFLOPS. The building block of Summit is the AC922 node, as shown in Fig. 2d. Each node contains two POWER9 processors and six NVIDIA Tesla V100 accelerators. Each processor is also connected via dual NVLINK bricks, each capable of a 25 GB/s bandwidth in each direction. The connections between each node utilize dual-port InfiniBand interconnects, employing a non-blocking fat-tree topology switching structure, with each port providing a bandwidth of 200 Gbps.

2.3 Motivations

Although previous works have successfully optimized the DeePMD-kit on these supercomputers and demonstrated excellent performance, a significant performance gap between the peak performance and achieved performance by DeePMD-kit. Therefore, it is important to model and analyze the performance of DeePMD-kit, respecting diverse physical systems and targeted supercomputers. However, there are still several challenges as follows.

1. *No performance model can accurately analyze the resource utilization of the DeePMD-kit.* DeePMD-kit

contains many irregular customized operators and complex MPI communication patterns. And its computational graph changes according to the simulated physical systems. Existing performance models only focus on the specific aspects of some applications and lack generality in profiling DeePMD-kit. The roofline-based models (Jia et al. 2013; Williams 2009; Zhang et al. 2017) can analyze the computational resource utilization of official TensorFlow operators in the DeePMD-kit on a single process after completing a simulation, but cannot analyze the MPI communication. The LogP-based model (Culler et al. 1993; Alexandrov et al. 1995) can characterize the network bandwidth and the latency but cannot characterize general operators.

2. *Current performance model cannot predict the simulation time and the achievable performance of the DeePMD-kit without real runs.* We need to profile the computational limits of the DeePMD-kit on the specific platform, including the achievable performance, time-to-solution, respecting to the physical system. MD simulations for large physical systems necessitate the whole machine resources, making the simulations expensive and time-consuming. Providing an accurate prediction can benefit this procedure. Simulating long timesteps for large physical systems consumes substantial computational resources. Therefore, it is crucial to analyze the

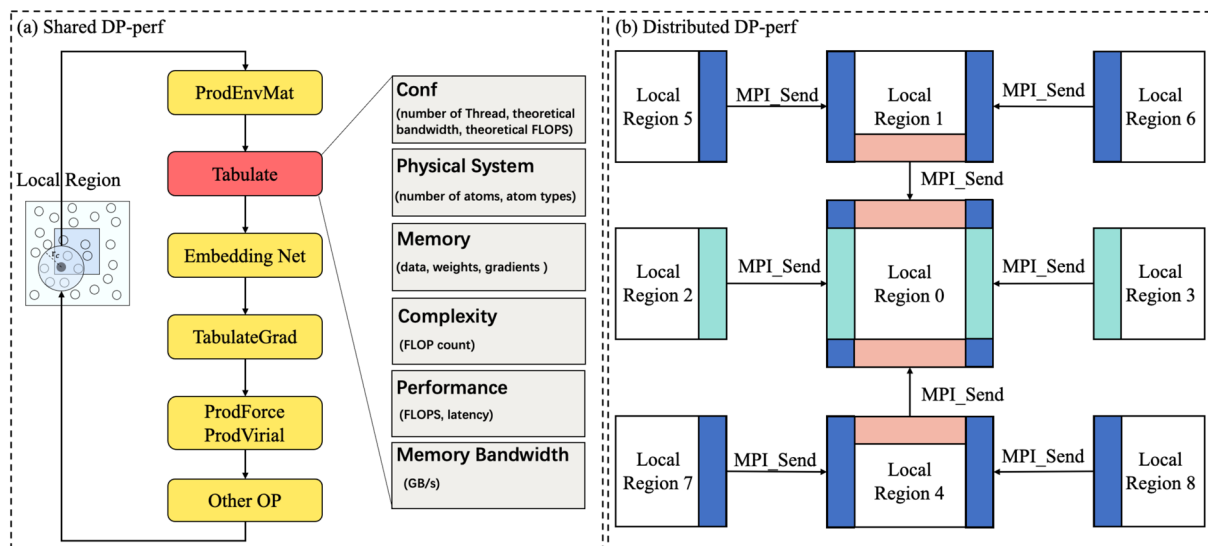


Fig. 3 Overview of the DP-perf model. **a** Shared DP-perf model. It characterizes the DeePMD-kit operators, including the customized operators and the official Tensorflow operators. **b** Distribute DP-perf model. It characterizes the communication pattern across MPI

processes. The communication occurs within the ghost regions (in colors) of local regions (without color) in each MPI process. Taking the two-dimensional box as an example, each process engages in synchronous communication along four directions

cost and determine the hardware/software configurations before launching these simulations.

3 Interpretable DeePMD-kit performance model

We propose an interpretable DeePMD-kit performance model called the DP-perf, which consists of two key components: shared DP-perf on a node and distributed DP-perf models across nodes, as shown in Fig. 3. The shared DP-perf model aims to analyze the execution time of each DeePMD-kit operator, and the distributed DP-perf model focuses on analyzing the MPI communication latency of the whole application. In practical scenarios, users primarily focus on achieving molecular dynamics simulations at high spatial and temporal scales. Consequently, we do not consider extremely small cases where the molecular system on each node is so small that it cannot fully occupy a cache line.

3.1 Derivation of arithmetic workload and memory access

Before constructing the performance model, it is essential to determine the total arithmetic workload N_{ops} and memory access M_{mem} of the DeePMD-kit operators. They are influenced by the characteristics of the molecular system and the algorithms.

For N_{ops} calculation, we take the `Tabulate` operator in DeePMD-kit as an example. As shown in Algorithm 1, the `Tabulate` operator approximates $D(\tilde{R})$ using a fifth-order polynomial. Initially, it iterates through $N \times N_n$ loops to process the neighboring atoms of all atoms in the physical system and determines the corresponding table weights (lines 1–5). Subsequently, it loops for K times to compute the polynomial result v and performs matrix multiplications with the embedding matrix R to obtain $D(\tilde{R})$ (lines 7–19). Therefore, the arithmetic workload of `Tabulate` operator is $O(N \times N_n \times K)$. For each inner loop, we can count 14 floating point operations. Finally, we construct the arithmetic workload as

$$N_{tab} = 14 \cdot N \cdot N_n \cdot K. \quad (4)$$

For M_{mem} calculation, we take the `Pad` operator in DeePMD-kit as an example. As shown in Algorithm 2, it applies zero padding along the selected dimension of the input tensor x . The total memory access is derived by adding the memory size of the input and the output tensor. As shown in Algorithm 2, the input size is $N \times M$ and the output size is $N \times N_n$. Then the total memory access (in bytes) is denoted as

$$M_{mem} = N \times M \cdot 8 + N \times N_n \cdot 8. \quad (5)$$

In DeePMD-kit, $M = N_n/2$. Then the total memory access is $M_{mem} = 12 \times N \times N_n$ bytes.

Algorithm 1 `Tabulate` operator

input : Table weight: $w \in \mathbb{R}^{N \times N_n \times K \times 6}$;
Smooth neighbor: $s \in \mathbb{R}^{N \times N_n}$;
Embedding mat: $R \in \mathbb{R}^{N \times N_n \times 4}$;
Number of atoms: N_{loc} ;
Number of neighbors: N_{nei} ;
Feature dimension: K ;
output: $\mathcal{D} \in \mathbb{R}^{N \times 4 \times K}$;

```

1 //Iterate over the number of atoms;
2 for  $i \leftarrow 0$  to  $N$  do
3   // Iterate over the neighbor atoms;
4   for  $j \leftarrow 0$  to  $N_n$  do
5      $idx \leftarrow \text{LocateTable}(s[i, j])$ ;
6     //Iterate over the tabulate feature;
7     for  $k \leftarrow 0$  to  $K$  do
8        $a_0 \leftarrow w[idx, k, 0]$ ;
9        $a_1 \leftarrow w[idx, k, 1]$ ;
10       $a_2 \leftarrow w[idx, k, 2]$ ;
11       $a_3 \leftarrow w[idx, k, 3]$ ;
12       $a_4 \leftarrow w[idx, k, 4]$ ;
13       $a_5 \leftarrow w[idx, k, 5]$ ;
14      //Calculate the polynomial;
15       $v \leftarrow a_0 + (a_1 + (a_2 + (a_3 +$ 
         $(a_4 + a_5 * s[i, j]) * s[i, j]) * s[i, j]) * s[i, j]) * s[i, j]$ ;
16       $\mathcal{D}[i, 0, k] \leftarrow v * R[i, j, 0]$ ;
17       $\mathcal{D}[i, 1, k] \leftarrow v * R[i, j, 1]$ ;
18       $\mathcal{D}[i, 2, k] \leftarrow v * R[i, j, 2]$ ;
19       $\mathcal{D}[i, 3, k] \leftarrow v * R[i, j, 3]$ ;
20    end
21  end
22 end
```

Algorithm 2 Pad operator

```

input : Pad feature:  $x \in \mathbb{R}^{N \times M}$ ;
        Number of atoms:  $N$ ;
        Dimensions before padding:  $M$ ;
        Number of neighbors:  $N_n$ ;
output:  $y \in \mathbb{R}^{N \times N_n}$ ;
1 //Iterate over the number of atoms;
2 for  $i \leftarrow 0$  to  $N$  do
3   // Iterate over the neighbor atoms;
4   for  $j \leftarrow 0$  to  $N_n$  do
5      $y \leftarrow \text{PadZero}(x, N, N_n, M)$ ;
6   end
7 end

```

3.2 DP-perf performance model

As illustrated in Sect. 3.1, the arithmetic workload and memory access of DeePMD-kit operators can be derived by the physical parameters including the MD parameters and the DeePMD-kit model configurations. Meanwhile, we collect the hardware parameters (i.e., machine configurations) to construct the DP-perf model. Table 2 provides the description of the parameters used to build the DP-perf model. Subsequently, we develop both shared DP-perf and distributed DP-perf models to estimate the execution time of each DeePMD-kit operator and the communication time across MPI processes.

3.2.1 Shared DP-perf

Shared DP-perf focuses on the execution time of the DeePMD-kit operators. Figure 3a presents its overview. It is constructed according to a straightforward derivation to calculate the peak performance achieved by the DeePMD-kit operators. The execution time of operators can be represented as follow:

$$t = \frac{N_{ops}}{N_{node} \times P_{achieved}} + \beta, \quad (6)$$

where the N_{ops} denotes the total arithmetic workload of the operator derived from the physical characteristics and the algorithm of the corresponding operator. β denotes the latency caused by the memory (de-)allocations and the kernel launching. And $P_{achieved}$ is the achievable peak performance, which can be denoted as $P_{achieved} = \alpha P$ (P is theoretical peak performance of the computing node). Then the execution time is denoted as

$$t = \frac{1}{\alpha} \frac{N_{ops}}{N_{node} \times P} + \beta, \quad (7)$$

where $\frac{1}{\alpha}$ can be obtained by applying linear regression.

Tensor manipulation operators such as Pad and Slice involve data format conversion in memory and do not include any computation. Therefore, we introduce the bandwidth equation to represent these operators, which is denoted as

$$t = \frac{M_{mem}}{N_{node} \times B_{achieved}} + \beta, \quad (8)$$

M_{mem} denotes the total memory access of the operator which can be derived from physical characteristics and the algorithm of the corresponding operator in Sect. 3.1. β

Table 2 Hardware and physical parameters for constructing DP-perf

	Parameters	Description
Hardware	P	Theoretical peak performance of the single node
	B_m	The memory bandwidth of a CPU
	B_n	The network bandwidth of the HPC
	L	Network latency
	N_{node}	Number of computing nodes
Molecular	N	Number of atoms in a physical system
	N_n	Number of neighbor atoms for each atom
	T	Number of atomic types in a physical system
	W	The number of ghost atoms per MPI process
Model	K	The dimension of the descriptor
	I_i	The input feature dimension of the i-th dense layer
	O_i	The output feature dimension of the i-th dense layer
	L	The number of the dense layers in fitting net

denotes the latency caused by the memory (de-)allocations and the kernel launching. And $B_{achieved}$ is the achievable memory bandwidth on a single node, which is denoted as $B_{achieved} = \alpha B_m$ (B_m is the peak memory bandwidth of the computing node). Then the execution time is denoted as

$$t = \frac{1}{\alpha} \frac{M}{N_{node} \times B_m} + \beta, \quad (9)$$

where $\frac{1}{\alpha}$ can be obtained by applying linear regression.

3.2.2 Distributed DP-perf

DeePMD-kit is used as a plugin embedded in LAMMPS, enabling large-scale simulations. Each MPI process of the DeePMD-kit follows the communication pattern of LAMMPS to update the atom positions. As shown in Fig. 3b, each process consists of the local region and the ghost region (colored). The local region is primarily used to calculate the interactions between atoms, while the ghost region is used to calculate the atomic force and execute data communication. The communication of LAMMPS is expressed as executing the MPI_Send request in the ghost area in each direction of the box. Figure 3b shows the communication of the ghost atom in the 2D box. Taking process 0 as an example, processes 2 and 3 send the owned atoms in their green regions to process 0 in the x-axis exchanging stage. Then in the y-axis exchanging stage,

processes 1 and 4 send atoms in their red regions to process 0, which includes ghost atoms they received in the x stage. Process 0 thus acquires all its ghost atoms; atoms in the solid blue corner regions are communicated twice before process 0 receives them. For a 3D box, the communication can be expressed as MPI_Send in six directions, and the communication volume is the number of atoms in the ghost region of the send direction. Based on this communication pattern, we construct the distributed DP-perf model to analyze the communication time between processes in large-scale MD simulations. We assume that the number of ghost atoms in each process remains unchanged and positively correlated with ghost region. Therefore, according to logP model (Hoeffler et al. 2010; Alexandrov et al. 1995; Hoeffler et al. 2007), the communication time is expressed as follows:

$$t_c = L \cdot (n_p - 1) + \alpha_c \sum_i^d \frac{W_i}{B_n}, \quad (10)$$

where the L denotes the sum of the latency and overlap of each process, B_n denotes the theoretical network bandwidth, α_c denotes the parameters related to the network bandwidth utilization, and W_i denotes the total number of ghost atoms in each direction. $\sum_i^d W_i$ denotes the total ghost atoms in each MPI process.

Table 3 DP-perf performance models, and the trained coefficients on four top supercomputers

Performance Equation	Tianhe-3F	New Sunway	Fugaku	Summit
$t_{env} = \alpha_1 \cdot \frac{472 \cdot N \cdot N_n}{N_{node} \cdot P} + \beta_1$	$\alpha_1 = 6.76$ $\beta_1 = 0.46$	$\alpha_1 = 165.42$ $\beta_1 = 0.21$	$\alpha_1 = 23.28$ $\beta_1 = 2.61$	$\alpha_1 = 13.94$ $\beta_1 = 1.47$
$t_{tab} = \alpha_2 \cdot \sum_{i=0}^T \sum_{j=0}^T \frac{14 \cdot N^i \cdot N_n^j \cdot K}{N_{node} \cdot P} + \beta_2$	$\alpha_2 = 3.86$ $\beta_2 = 1.65$	$\alpha_2 = 43.13$ $\beta_2 = 0.18$	$\alpha_2 = 5.20$ $\beta_2 = 1.02$	$\alpha_2 = 1.09$ $\beta_2 = 1.52$
$t_{tabg} = \alpha_3 \cdot \sum_{i=0}^T \sum_{j=0}^T \frac{46 \cdot N^i \cdot N_n^j \cdot K}{N_{node} \cdot P} + \beta_3$	$\alpha_3 = 1.19$ $\beta_3 = 0.76$	$\alpha_3 = 42.73$ $\beta_3 = 0.16$	$\alpha_3 = 9.41$ $\beta_3 = 3.00$	$\alpha_3 = 3.35$ $\beta_3 = 2.94$
$t_{dense} = \alpha_4 \cdot \sum_{i=0}^T \sum_{j=0}^L \frac{2 \cdot N^i \cdot J_j \cdot O_j}{N_{node} \cdot P} + \beta_4$	$\alpha_4 = 3.50$ $\beta_4 = 0.10$	$\alpha_4 = 16.55$ $\beta_4 = 0.17$	$\alpha_4 = 1.33$ $\beta_4 = 1.62$	$\alpha_4 = 1.79$ $\beta_4 = 0.01$
$t_{denseg} = \alpha_5 \cdot \sum_{i=0}^T \sum_{j=0}^L \frac{N^i \cdot J_j \cdot O_j}{N_{node} \cdot P} + \beta_5$	$\alpha_5 = 45.41$ $\beta_5 = 0.01$	$\alpha_5 = 16.80$ $\beta_5 = 0.18$	$\alpha_5 = 3.14$ $\beta_5 = 2.19$	$\alpha_5 = 4.54$ $\beta_5 = 0.31$
$t_{force} = \alpha_6 \cdot \frac{48 \cdot N \cdot N_n}{N_{node} \cdot P} + \beta_6$	$\alpha_6 = 15.84$ $\beta_6 = 0.05$	$\alpha_6 = 161.23$ $\beta_6 = 0.16$	$\alpha_6 = 13.67$ $\beta_6 = 0.06$	$\alpha_6 = 1.81$ $\beta_6 = 0.11$
$t_{virial} = \alpha_7 \cdot \frac{152 \cdot N \cdot N_n}{N_{node} \cdot P} + \beta_7$	$\alpha_7 = 2.29$ $\beta_7 = 0.04$	$\alpha_7 = 28.78$ $\beta_7 = 0.02$	$\alpha_7 = 4.47$ $\beta_7 = 0.13$	$\alpha_7 = 1.99$ $\beta_7 = 0.10$
$t_{tanh} = \alpha_8 \cdot \sum_{i=0}^T \sum_{j=0}^L \frac{5 \cdot N^i \cdot O_j}{N_{node} \cdot P} + \beta_8$	$\alpha_8 = 11.61$ $\beta_8 = 0.02$	$\alpha_8 = 11.73$ $\beta_8 = 0.04$	$\alpha_8 = 26.75$ $\beta_8 = 0.81$	$\alpha_8 = 33.62$ $\beta_8 = 0.01$
$t_{pad} = \alpha_9 \cdot \frac{12 \cdot N \cdot N_n}{B_m \cdot N_{node}} + \beta_9$	$\alpha_9 = 1.40$ $\beta_9 = 1.70$	$\alpha_9 = 1.28$ $\beta_9 = 0.17$	$\alpha_9 = 1.05$ $\beta_9 = 0.19$	$\alpha_9 = 1.48$ $\beta_9 = 1.47$
$t_{slice} = \alpha_{10} \cdot \frac{12 \cdot N \cdot N_n}{B_m \cdot N_{node}} + \beta_{10}$	$\alpha_{10} = 1.54$ $\beta_{10} = 1.69$	$\alpha_{10} = 1.31$ $\beta_{10} = 0.17$	$\alpha_{10} = 2.00$ $\beta_{10} = 0.05$	$\alpha_{10} = 1.10$ $\beta_{10} = 2.12$
$t_c = L(n_p - 1) + \alpha_c \sum_i^d \frac{W_i}{B_n}$	$\alpha_c = 4.38$ $L = 6.95 \times 10^{-6}$	$\alpha_c = 9.38$ $L = 5.21 \times 10^{-8}$	$\alpha_c = 2.02$ $L = 4.67 \times 10^{-8}$	$\alpha_c = 1.29$ $L = 1.68 \times 10^{-6}$

3.3 Overall MD latency prediction

Based on the shared and distributed DP-perf model, we create the tailored performance models for each DeePMD-kit operator and the communication pattern. Table 3 shows the detailed performance equations of the DP-perf model, and the trained coefficients on four top supercomputers. Each shared DP-perf model has two parameters: α_n and β_n . For customized DeePMD-kit operators, α_n is related to the peak performance utilization, and the lower value indicates the higher peak performance utilization. For tensor manipulation operators, α_n is related to the memory bandwidth utilization, and the lower value indicates the higher bandwidth utilization. Additionally, β_n is related to the latency of memory operations and kernel launching. It includes the latency for memory copy, data loading, and launching kernels. α_c and L were the parameters in the Distribute DP-perf model, where α_c is related to the utilization of the network bandwidth and the L is related to the network latency. Then, we use the training logs on four top supercomputers to fit the parameters. We can easily obtain the computation time and MPI communication time from output logs in LAMMPS format. However, the computation time of each operator is counted through the `MPI_wtime()` code. Finally, we fit the parameters according to these logs and list them in Table 3. The overall time of a DeePMD-kit simulation is denoted as:

$$t_{\text{overall}} = t_{\text{env}} + t_{\text{tab}} + t_{\text{tabg}} + t_{\text{dense}} + t_{\text{denseg}} + t_{\text{force}} + t_{\text{virial}} + t_{\text{tanh}} + t_{\text{pad}} + t_{\text{slice}} + t_c. \quad (11)$$

4 Evaluation

4.1 Experimental setup

We use two systems as the benchmarks for evaluation, which are the water system and the copper system. Both systems have been trained and evaluated on previous works (Zhang et al. 2018, 2018, 2020). We selected the scaling results from four supercomputers to train the DP-perf model. Specifically, the raw data on Summit and Fugaku is obtained from the already published paper (Guo et al. 2022), and the raw data on Tianhe-3F and new Sunway are derived by our real runs, based on the open-source repositories.¹

We construct the training dataset by collecting weak and strong scaling logs. For the strong scaling simulations, we use 32.5/14.9/8.3/41.5 million atoms of water systems and

5.9/2.5/2.2/13.5 million atoms of copper systems to saturate the memory of 50 nodes and gradually increase the computing resource to 4600 nodes on Tianhe-3F, new Sunway, Fugaku, and Summit. For the weak scaling simulations, we use 17.1/7.6/97.3/243.8 million atoms of the water system and 7.1/3.4/67.6/212.5 million atoms of the copper system to saturate 256/256/621/575 nodes of the total machine memory, then gradually increase the system size and the number of nodes to the entire machine and measure the achieved peak performance of simulations on these four platforms. Due to the limitation of accessible resources, we only measure the weak scaling results on 2048/90,000/9936 nodes on Tianhe-3F, new Sunway, and Fugaku respectively. After collecting the evaluation results, we construct the datasets with 26/31/21/32 samples on Tianhe-3F, new Sunway, Fugaku, and Summit, respectively. We randomly split the dataset from Tianhe-3F, Fugaku, new Sunway, and Summit into the training dataset and test dataset with a ratio of 4:1.

4.2 Fitting accuracy on known dataset

To evaluate the fitting accuracy on known dataset, we use the training dataset on Tianhe-3F, new Sunway, Fugaku, and Summit supercomputers to fit the execution time. Figure 4 shows the detailed experiment results, where the x-axis denotes the actual execution time, and the y-axis denotes the fitted time by the DP-perf model. Additionally, we draw the $y = x$ reference line (the red line) in the figure. As shown in Fig. 4a, the fitted MD time points exhibit a uniform distribution surrounding the $y = x$ line, indicating the linear correlation with the actual executions. Through Pearson correlation analysis, the Pearson coefficient reaches 0.9941 on Tianhe-3F, 0.9925 on Fugaku, 0.999 on the new Sunway, and 0.966 on Summit, which shows a significant linear correlation. We then evaluate the mean absolute percentage error (MAPE) between the fitted MD time and the actual MD time. On Tianhe-3F, DP-perf has an average MAPE of 5.7%. On the new Sunway, DP-perf has an average MAPE of 8.1%. On Fugaku, DP-perf has an average MAPE of 14.3 %. On the Summit, DP-perf has an average MAPE of 13.5%. We analyze the accuracy of the shared DP-perf and distributed DP-perf models, as illustrated in Fig. 4b and c. Both models showed high accuracy, with the shared DP-perf model achieving Pearson coefficients of 0.997 on Tianhe-3F, 0.999 on the new Sunway, 0.943 on Fugaku, and 0.937 on Summit. The distributed DP-perf model achieved Pearson coefficients of 0.987 on Tianhe-3F, 0.991 on the new Sunway, 0.829 on Fugaku, and 0.833 on Summit. These results in Fig. 4 demonstrate that the fitted parameters of the DP-perf model can accurately represent peak performance utilization, memory bandwidth utilization, and network bandwidth utilization at a low error. We further evaluate the MAPE of each operator on these four supercomputers in Table 4.

¹ <https://bitbucket.org/xiangyumengupc/deepmd-kit-tianhe>, <https://bitbucket.org/xiangyumengupc/deepmd-kit-sunway>.

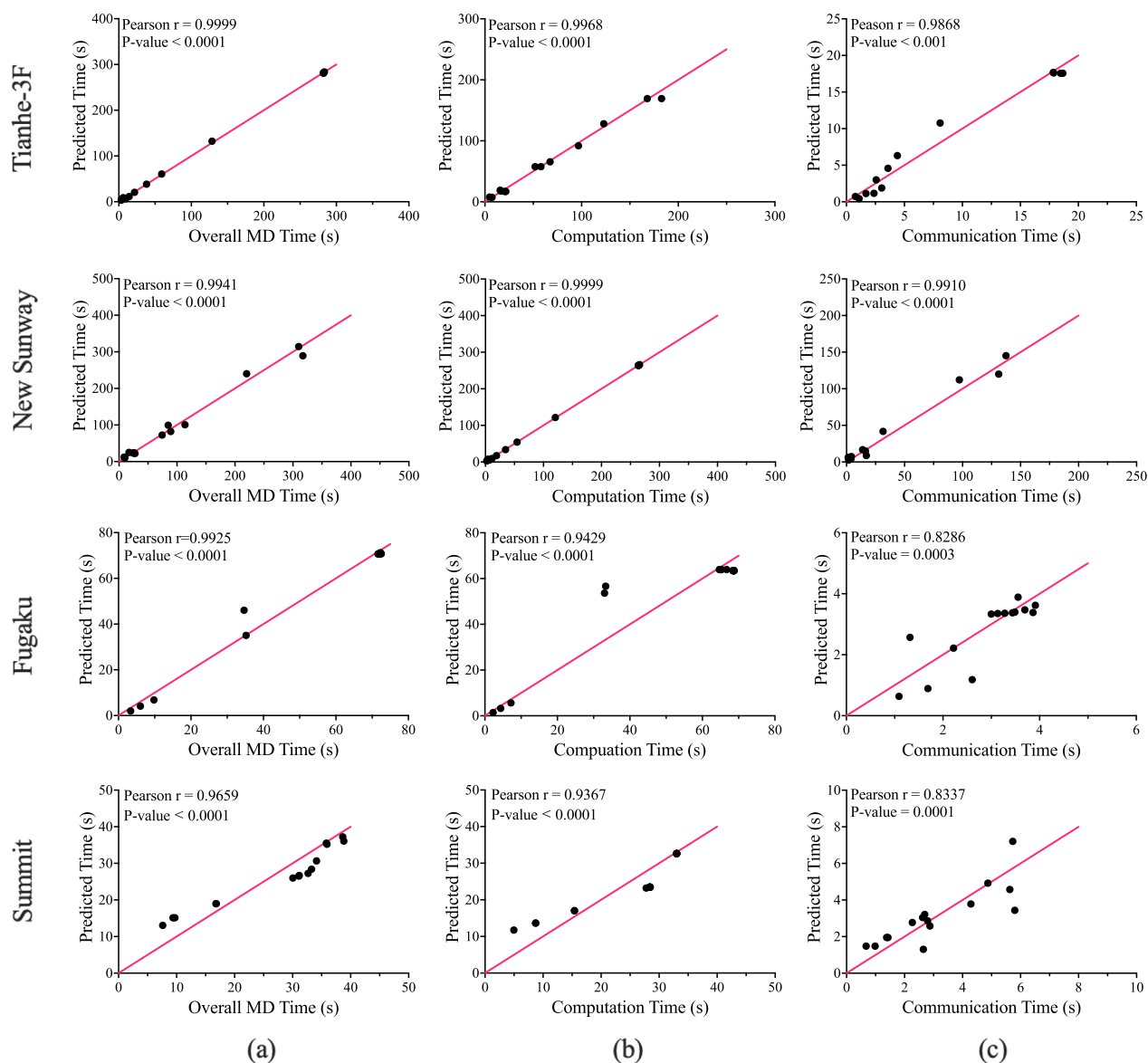


Fig. 4 Accuracy of simulation time fitted by DP-perf model for copper and water systems. The x-axis represents the actual MD time, and the y-axis represents the fitted time. The red line is a visual guide for

$y = x$. **a** Comparison of overall MD time. **b** Comparison of computation time using shared DP-perf. **c** Comparison of communication time using distributed DP-perf

Table 4 MAPE of fitting for each operator on four supercomputers

	t_{env}	t_{tab}	t_{tag}	t_{dense}	t_{deseg}	t_{force}	t_{virial}	t_{tanh}	t_{pad}	t_{slice}
Tianhe-3F	11.0%	16.2%	16.7%	6.4%	9.8%	14.6%	11.8%	7.7%	6.8%	7.9%
new Sunway	7.0%	11.0%	8.7%	4.5%	1.3%	3.0%	12.2%	1.0%	6.4%	7.1%
Fugaku	1.0%	15.3%	12.5%	18.6%	11.4%	6.2%	17.0%	3.1%	19.2%	19.0%
Summit	15.6%	8.0%	20.0%	11.9%	19.4%	8.1%	19%	16.6%	2.0%	19.4%

According to the results, each operator achieves a low error rate of less than 20%. The main reason is that the DP-perf only considers peak performance, memory bandwidth, and

network bandwidth as the machine characteristics, but cannot consider specific characteristics among different

Table 5 Accuracy of the overall time prediction on four supercomputers using DP-perf

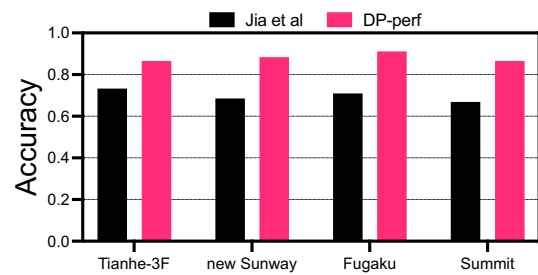
Machine	Pearson coef	MAPE	R^2
Tianhe-3F	0.990	13.5%	0.955
New Sunway	0.986	11.6%	0.952
Fugaku	0.997	8.9%	0.969
Summit	0.978	13.5%	0.830

supercomputers like network topology, and bandwidth contention, impacting the accuracy.

4.3 Prediction accuracy on unknown dataset

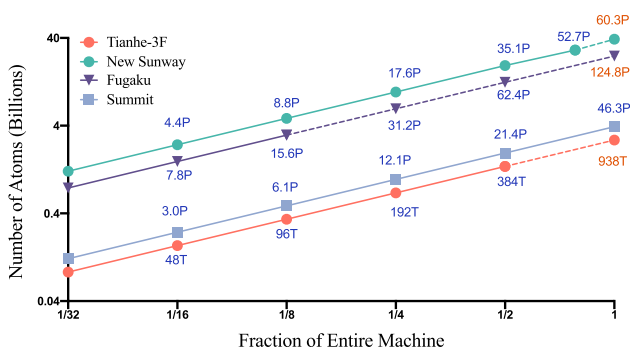
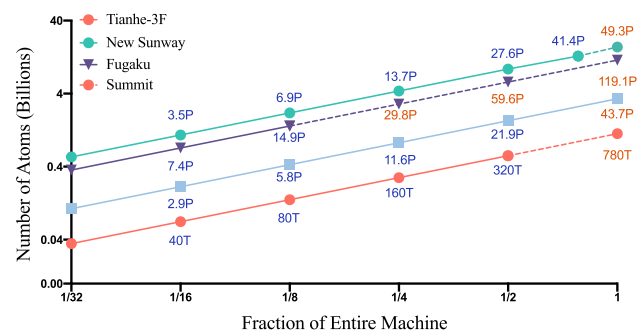
We further carry out a predictive evaluation to analyze the accuracy of overall MD time prediction. We use the train dataset to train the DP-perf model and use the test dataset to verify its accuracy. To avoid randomness and the fitting uncertainties of the DP-perf, we repeat the above steps 5 times and calculate the average metrics. Table 5 shows our results. We can observe that the MD simulation time predicted by DP-perf maintains a very low average MAPE and exhibits a strong linear correlation with the actual tested results, which indicates that DP-perf can predict MD simulation and achievable peak performance of DeepMD-kit with high accuracy.

Meanwhile, we further evaluate the prediction accuracy compared to the state-of-the-art performance model (Jia et al. 2013) in Fig. 6. We find the DP-perf outperformed on four supercomputers, which denotes that the DP-perf can accurately describe the utilization of machine computing resources and predict execution time and computational performance.

**Fig. 6** Prediction accuracy of DP-perf compared to the state-of-the-art performance model in Jia et al. (2013)

4.4 Prediction of achievable peak performance

In Sect. 4.3, the trained DP-perf model can accurately predict the execution time of DeepMD-kit. Based on the predicted time and the DeepMD-kit workload, we can calculate the corresponding achievable peak performance of this MD simulation for given physical systems. According to the previous weak scaling logs, we have measured 1.2/29/1.6/3.9 billion atoms water system and 0.8/13/1.2/3.4 billion atoms copper system on 2048/90,000/9936/4600 nodes on Tianhe-3F, new Sunway, Fugaku, and Summit, respectively. For the rest results scaling to the whole machine, we further predict the achievable peak performance using DP-perf. Figure 5 shows the prediction of weak scaling results on these platforms, where the solid line represents the actual result in our previous evaluations, and the dashed line represents the predicted results based on the DP-perf. Note that previous works have performed the weak scaling evaluation on the whole Summit machine, but we do not need that with the accurate prediction of DP-perf. According to the results, we predict that the optimized DeepMD-kit can achieve 24/3/38/3.9 billion atoms water simulation with 938T/124.8P/60.3P/46.3P FLOPS on the whole machine of the Tianhe-3F, new Sunway, Fugaku, and Summit

**(a) Water System****(b) Copper System****Fig. 5** Achievable peak performance prediction using DP-perf. The solid line represents the actual peak performance result in previous works. The dashed line represents the predicted peak performance result based on DP-perf

Summit, respectively. Meanwhile, the optimized DeePMD-kit can achieve 780T/119.1P/49.3P/43.7P FLOPS for 1.5/13/17.4/3.4 billion atoms copper system on the whole machine of these supercomputers.

5 Related work

Performance optimization of DeePMD-kit DeePMD-kit (Wang et al. 2018) has increased the simulation temporal and spatial by orders of magnitude while maintaining the *ab-initio* accuracy. The GPU version of DeePMD-kit (Jia et al. 2020; Lu et al. 2021) optimized on Summit has achieved 127 million atoms AIMD simulation. DP-compress model (Lu et al. 2022) has reduced the computational workload and memory using the Weierstrass approximation method to boost the performance of DeePMD-kit. Based on DP-compress, we have extended the performance of DeePMD-kit to achieve 1.5 billion atoms AIMD simulation with 7.8 PFLOPS on 9936 Fugaku nodes (around 6% of the whole machine), and 1 billion atoms simulation with 46 PFLOPS on Summit (whole machine) (Guo et al. 2022). Recently, we have performed novel mixed precision schemes to reach 1 billion atoms AIMD simulation with 384 TFLOPS on 2048 Tianhe-3F (half of the whole machine) and 29 billion atoms simulation with 58 PFLOPS on 90,000 new Sunway nodes (80% of the whole machine). Due to the limited available computing resources, we can not evaluate the DeePMD-kit performance of the whole machine. Moreover, there is no effective method to measure the execution time of the DeePMD-kit and predict its computational performance on the whole machine.

Performance models Performance models can quantitatively measure the computing efficiency of applications on different architectures. Roofline-based models (Williams 2009; Lo et al. 2015; Marques et al. 2021) provide instructive analysis to measure the memory bandwidth bottleneck. Execution–Cache–Memory (ECM) models (Treibig and Hager 2009; Stengel et al. 2015) consider the contributions from the complete memory hierarchy and the overlapping of execution and data transfers. Performance models adopted in auto-tuning approaches, such as IA-SpGEMM (Xie et al. 2019) and AlphaSparse (Du et al. 2022), help to determine the best format, algorithm, and etc. for arbitrary sparse matrix. The above performance models can describe the bottleneck in computing resources on simple loop codes (i.e. stencil and matrix multiplication) and help to improve the performance. However, measuring DeePMD-kit applications involves numerous irregular customized DeePMD-kit operators and MPI communications. DP-perf addresses the challenge by leveraging the characteristics of physical molecular systems and the DeePMD-kit algorithm to construct arithmetic workloads, memory access patterns, and

network communications. This approach provides a quick and accurate method to measure the resource utilization and the achievable peak performance of DeePMD-kit during large-scale AIMD simulations.

6 Conclusion

In this paper, we propose an interpretable performance model called DP-perf. This model characterizes the computation and communication pattern of the DeePMD-kit at the operator level, according to the machine configurations and the physical MD systems. DP-perf can obtain the accurate fitting and prediction to analyze the performance utilization of each DeePMD-kit operator and the communications. Meanwhile, DP-perf can accurately predict the MD simulation time and the achievable peak performance. DP-perf can offer valuable insights for the design and application innovation of new E-class supercomputers and bridging the performance gap between many-core supercomputers and *ab initio* molecular dynamics. Currently, DP-perf model performance is based on the peak performance, memory bandwidth, and network bandwidth of the machine, without adequately considering the impact of the cache-line on the performance of DeePMD-kit. In our future work, we will continue to refine the DP-perf model by considering the network topology, bandwidth conflict, and cache prefetching to further enhance the accuracy of DP-perf.

Acknowledgements This work is supported by the following funding: the Strategic Priority Research Program of Chinese Academy of Sciences (No. XDB0500102), National Science Foundation of China (No. 61972416, T2125013 and 92270206), China National Postdoctoral Program for Innovative Talents (No. BX20240383), the Natural Science Foundation of Shandong Province (No. ZR2022LZH009), GHfound C (No. 202407035455), and National Key R&D Program of China (No. 2021YFA1000103-3). The numerical calculations in this study are carried out on the new Sunway, TianHe-3F, and Fugaku Supercomputers. Xun Wang, Mingzhen Li, and Weile Jia are the corresponding authors.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alexandrov, A., Ionescu, M.F., Schauser, K.E., Scheiman, C.: Loggp: incorporating long messages into the logp model—one step closer towards a realistic model for parallel computation. In: Proceedings of the Seventh Annual ACM Symposium on Parallel Algorithms and Architectures, pp. 95–105. (1995)
- Batzner, S., Musaelian, A., Sun, L., Geiger, M., Mailoa, J.P., Kornbluth, M., Molinari, N., Smidt, T.E., Kozinsky, B.: E (3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nat. Commun.* **13**(1), 2453 (2022)
- Behler, J., Parrinello, M.: Generalized neural-network representation of high-dimensional potential-energy surfaces. *Phys. Rev. Lett.* **98**(14), 146401 (2007)
- Chen, W., Dong, X., Chen, H., Wang, Q., Yu, X., Zhang, X.: Performance evaluation of convolutional neural network on tianhe-3 prototype. *J. Supercomput.* 12647–12665 (2021). <https://doi.org/10.1007/s11227-021-03759-8>
- Culler, D., Karp, R., Patterson, D., Sahay, A., Schauser, K.E., Santos, E., Subramonian, R., Von Eicken, T.: Logp: towards a realistic model of parallel computation. In: Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, pp. 1–12. (1993)
- Du, Z., Li, J., Wang, Y., Li, X., Tan, G., Sun, N.: Alphaspase: generating high performance SPMV codes directly from sparse matrices. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–15. IEEE, New York (2022)
- Friederich, P., Häse, F., Proppe, J., Aspuru-Guzik, A.: Machine-learned potentials for next-generation matter simulations. *Nat. Mater.* **20**(6), 750–761 (2021)
- Gan, X., Zhang, Y., Wang, R., Li, T., Xiao, T., Zeng, R., Liu, J., Lu, K.: Tianhegraph: customizing graph search for graph500 on Tianhe supercomputer. *IEEE Trans. Parallel Distrib. Syst.* **33**(4), 941–951 (2021)
- Gao, J., Zheng, F., Qi, F., Ding, Y., Li, H., Lu, H., He, W., Wei, H., Jin, L., Liu, X., et al.: Sunway supercomputer architecture towards exascale computing: analysis and practice. *Sci. China Inf. Sci.* **64**(4), 141101 (2021)
- Guo, Z., Lu, D., Yan, Y., Hu, S., Liu, R., Tan, G., Sun, N., Jiang, W., Liu, L., Chen, Y., et al.: Extending the limit of molecular dynamics with ab initio accuracy to 10 billion atoms. In: Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, pp. 205–218 (2022)
- Hoefler, T., Lichei, A., Rehm, W.: Low-overhead loggp parameter assessment for modern interconnection networks. In: 2007 IEEE International Parallel and Distributed Processing Symposium, pp. 1–8. IEEE, New York (2007)
- Hoefler, T., Schneider, T., Lumsdaine, A.: Loggopsim: simulating large-scale applications in the loggops model. In: Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing, pp. 597–604. (2010)
- Hu, W., An, H., Guo, Z., Jiang, Q., Qin, X., Chen, J., Jia, W., Yang, C., Luo, Z., Li, J., et al.: 2.5 million-atom ab initio electronic-structure simulation of complex metallic heterostructures with DGDFT. In: 2022 SC22: International Conference for High Performance Computing, Networking, Storage and Analysis (SC), pp. 48–60. IEEE Computer Society, New York (2022)
- Jia, W., Cao, Z., Wang, L., Fu, J., Chi, X., Gao, W., Wang, L.-W.: The analysis of a plane wave pseudopotential density functional theory code on a GPU machine. *Comput. Phys. Commun.* **184**(1), 9–18 (2013)
- Jia, W., Wang, H., Chen, M., Lu, D., Lin, L., Car, R., Weinan, E., Zhang, L.: Pushing the limit of molecular dynamics with ab initio accuracy to 100 million atoms with machine learning. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. IEEE, New York (2020)
- Lin, L., Lu, J., Ying, L., Car, R., E, W.: Fast algorithm for extracting the diagonal of the inverse matrix with application to the electronic structure analysis of metallic systems. *Comm. Math. Sci.* **7**, 755 (2009)
- Lo, Y.J., Williams, S., Van Straalen, B., Ligocki, T.J., Cordery, M.J., Wright, N.J., Hall, M.W., Olikar, L.: Roofline model toolkit: a practical tool for architectural and program analysis. In: High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation: 5th International Workshop, PMBS 2014, New Orleans, LA, USA, November 16, 2014. Revised Selected Papers 5, pp. 129–148. Springer, Berlin (2015)
- Lu, D., Wang, H., Chen, M., Lin, L., Car, R., Weinan, E., Jia, W., Zhang, L.: 86 pflops deep potential molecular dynamics simulation of 100 million atoms with ab initio accuracy. *Comput. Phys. Commun.* **259**, 107624 (2021)
- Lu, D., Jiang, W., Chen, Y., Zhang, L., Jia, W., Wang, H., Chen, M.: DP compress: a model compression scheme for generating efficient deep potential models. *J. Chem. Theory Comput.* **18**(9), 5559–5567 (2022)
- Marques, D., Ilic, A., Sousa, L.: Mansard roofline model: reinforcing the accuracy of the roofs. *ACM Trans. Model. Perform. Eval. Comput. Syst.* **6**(2), 1–23 (2021)
- Richards, D.F., Glosli, J.N., Chan, B., Dorr, M.R., Draeger, E.W., Fattebert, J.-L., Krauss, W.D., Spelce, T., Streitz, F.H., Surh, M.P., et al.: Beyond homogeneous decomposition: scaling long-range forces on massively parallel systems. In: Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis, pp. 1–12 (2009)
- Salomon-Ferrer, R., Gotz, A.W., Poole, D., Le Grand, S., Walker, R.C.: Routine microsecond molecular dynamics simulations with AMBER GPUs. 2. Explicit solvent particle mesh Ewald. *J. Chem. Theory Comput.* **9**(9), 3878–3888 (2013)
- Schade, R., Kenter, T., Elgabarty, H., Lass, M., Schütt, O., Lazzaro, A., Pabst, H., Mohr, S., Hutter, J., Kühne, T.D., et al.: Towards electronic structure-based ab-initio molecular dynamics simulations with hundreds of millions of atoms. *Parallel Comput.* **111**, 102920 (2022)
- Schütt, K., Unke, O., Gastegger, M.: Equivariant message passing for the prediction of tensorial properties and molecular spectra. In: International Conference on Machine Learning, pp. 9377–9388. PMLR, Westminster (2021)
- Stengel, H., Treibig, J., Hager, G., Wellein, G.: Quantifying performance bottlenecks of stencil computations using the execution-cache-memory model. In: Proceedings of the 29th ACM on International Conference on Supercomputing, pp. 207–216. (2015)
- Tan, G., Sun, N., Gao, G.R.: Improving performance of dynamic programming via parallelism and locality on multicore architectures. *IEEE Trans. Parallel Distrib. Syst.* **20**(2), 261–274 (2008)
- Thompson, A.P., Aktulga, H.M., Berger, R., Bolintineanu, D.S., Brown, W.M., Crozier, P.S., Veld, P.J., Kohlmeyer, A., Moore, S.G., Nguyen, T.D., et al.: LAMMPS—a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Comput. Phys. Commun.* **271**, 108171 (2022)
- Treibig, J., Hager, G.: Introducing a performance model for bandwidth-limited loop kernels. In: International Conference on Parallel Processing and Applied Mathematics, pp. 615–624. Springer, Berlin (2009)
- Unke, O.T., Chmiela, S., Sauceda, H.E., Gastegger, M., Poltavsky, I., Schütt, K.T., Tkatchenko, A., Müller, K.-R.: Machine learning force fields. *Chem. Rev.* **121**(16), 10142–10186 (2021)
- Van Duin, A.C., Dasgupta, S., Lorant, F., Goddard, W.A.: Reaxff: a reactive force field for hydrocarbons. *J. Phys. Chem. A* **105**(41), 9396–9409 (2001)

- Wang, H., Zhang, L., Han, J., Weinan, E.: Deepmd-kit: a deep learning package for many-body potential energy representation and molecular dynamics. *Comput. Phys. Commun.* **228**, 178–184 (2018)
- Wang, R., Lu, K., Chen, J., Zhang, W., Li, J., Yuan, Y., Lu, P., Huang, L., Li, S., Fan, X.: Brief introduction of Tianhe exascale prototype system. *Tsinghua Sci. Technol.* **26**(3), 361–369 (2020)
- Wei, J., Zhang, X., Ji, Z., Li, J., Wei, Z.: Deploying and scaling distributed parallel deep neural networks on the Tianhe-3 prototype system. *Sci. Rep.* **11**(1), 20244 (2021)
- Williams, S.: Roofline: an insightful visual performance model for floating-point programs and multicore. *ACM Commun.* **16**, 65–76 (2009). <https://doi.org/10.1145/1498765.1498785>
- Xie, Z., Tan, G., Liu, W., Sun, N.: Ia-spgemm: an input-aware auto-tuning framework for parallel sparse matrix-matrix multiplication. In: *Proceedings of the ACM International Conference on Supercomputing*, pp. 94–105. (2019)
- Yan, Y., Li, H., Zhao, T., Wang, L., Shi, L., Liu, T., Tan, G., Jia, W., Sun, N.: 10-million atoms simulation of first-principle package LS3DF on Sugon supercomputer. *J. Comput. Sci. Technol.* **39**(1), 45–62 (2023)
- You, X., Yang, H., Luan, Z., Liu, Y., Qian, D.: Performance evaluation and analysis of linear algebra kernels in the prototype tianhe-3 cluster. In: *Asian Conference on Supercomputing Frontiers*, pp. 86–105. Springer, Berlin (2019)
- Zhang, X., Tan, G., Xue, S., Li, J., Zhou, K., Chen, M.: Understanding the GPU microarchitecture to achieve bare-metal performance tuning. In: *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 31–43. (2017)
- Zhang, L., Han, J., Wang, H., Car, R., Weinan, E.: Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics. *Phys. Rev. Lett.* **120**(14), 143001 (2018a)
- Zhang, L., Han, J., Wang, H., Saidi, W., Car, R., et al.: End-to-end symmetry preserving inter-atomic potential energy model for finite and extended systems. In: *Advances in Neural Information Processing Systems*, vol. 31. (2018b)
- Zhang, Y., Wang, H., Chen, W., Zeng, J., Zhang, L., Wang, H., Weinan, E.: Dp-gen: a concurrent learning platform for the generation of reliable deep learning based potential energy models. *Comput. Phys. Commun.* **253**, 107206 (2020)
- Zhang, L., Wang, H., Muniz, M.C., Panagiotopoulos, A.Z., Car, R., et al.: A deep potential model with long-range electrostatic interactions. *J. Chem. Phys.* (2022). <https://doi.org/10.1063/5.0083669>
- Zhu, Q., Luo, H., Yang, C., Ding, M., Yin, W., Yuan, X.: Enabling and scaling the HPCG benchmark on the newest generation Sunway supercomputer with 42 million heterogeneous cores. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '21*. Association for Computing Machinery, New York (2021)